

Building Effective Agents: A Short Hands-On Tutorial

Peyman Kor

2025-01-06

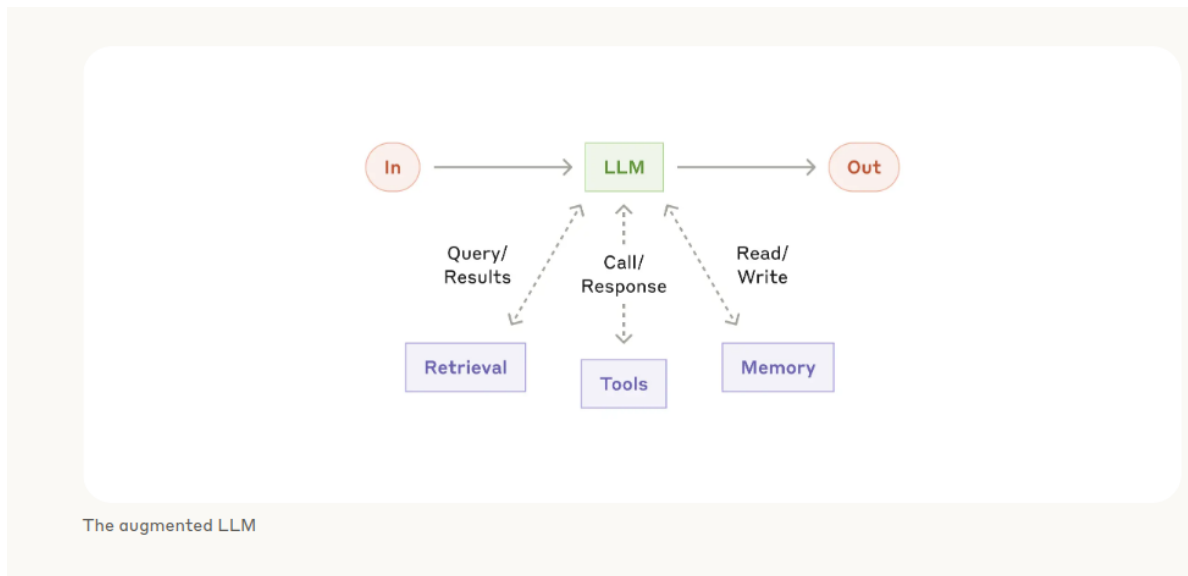


Figure 1: The concept of “Augmented LLM” introduced in Anthropic report - source: [Building the Effective Agent](#)

So here we want to review the latest report from the Anthropic named [Building the Effective Agent](#). The key message of the report is that when building agents, we should focus on the following principles:

- Maintain **simplicity** in your agent’s design.
- Prioritize **transparency** by explicitly showing the agent’s planning steps.

These two messages resonate very well with me because before I was a lot into the which packages to use, like a LangGraph and LangChain and different packages. Though I’m not

against working with the packages, but I think that for a starting point it's a best to use a simple Python code to build a simple agentic workflow. That's the best starting point.

The report also nicely distinguishes between **workflows** and **agents**. Workflows orchestrate predefined code paths, while agents dynamically direct their own resources and tool usage. Here, I am focusing on the workflows. As I believe that the workflows are the key building block for effective agents.

The report outlines five agent workflows:

- **Workflow 1: Prompt-Chaining**
- **Workflow 2: Parallelization**
- **Workflow 3: Routing**
- **Workflow 4: Evaluator-Optimizer**

For each of these workflow, I will start with a short description of the workflow and then we'll build up on that description the required function calling Python code of it and then the workflow section will end with the example to give a hands on example of how to implement the workflow.

LLM Call Function from Groq

In this report I will use the [Groq](#) as the LLM provider. So the way is that every function call that we will wake, will go to the [Groq](#) and then will receive the reply from the [Groq](#) LLM models. To do that you need to have the API access and you can find it in this [page](#) with this [guide](#) on how to get your API case. After getting your API case you're good to go to implement the workflows.

```
from IPython.core.display import display, HTML
display(HTML("<style>.output pre { white-space: pre-wrap; word-wrap: break-word; }</style>"))
```

```
/tmp/ipykernel_48361/3466160082.py:1: DeprecationWarning: Importing display from IPython.core
from IPython.core.display import display, HTML
```

<IPython.core.display.HTML object>

```
import os
from groq import Groq

os.environ["GROQ_API_KEY"] = "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"

client = Groq()
```

```

    api_key=os.environ.get("GROQ_API_KEY"), # This is the default and can be omitted
)

```

So in the previous code we generate the **client** class so then this client class has a “chat completion create” that that we can give the prompt which is a content and also specify the model and then we can get a reply.

```

chat_completion = client.chat.completions.create(
    messages=[
        {
            "role": "user",
            "content": "Explain the importance of low latency LLMs",
        }
    ],
    model="llama3-8b-8192",
)

```

For ease of use now I will define the `llm_call` function which only takes the prompt and give that prompt to the “llama3-70b-8192” model and I get a response from the as a return from output of the function. We will use `llm_call` function throughout this notebook.

```

def llm_call(prompt: str) -> str:

    chat_completion = client.chat.completions.create(
        messages=[
            {
                "role": "user",
                "content": prompt,
            }
        ],
        model="llama3-70b-8192",
    )
    return chat_completion.choices[0].message.content

```

Here we can just use this function as an example to see if everything is working.

```

import textwrap

task_prompt = "Explain me briefly the agentic AI concept"
response = llm_call(prompt=task_prompt)

```

```
wrapped_response = textwrap.fill(response, width=80) # Adjust width as needed
print(wrapped_response)
```

A great topic! Agentic AI is a concept in Artificial Intelligence (AI) that refers to autonomous systems that can act on their own behalf, making decisions and taking actions based on their own goals, motivations, and preferences. These AI systems are designed to be self-directed, proactive, and adaptable, rather than simply reacting to external stimuli or following pre-programmed rules. The key characteristics of agentic AI include: 1. **Autonomy**: The ability to make decisions and act independently, without human intervention. 2. **Agency**: The ability to influence its own behavior and the behavior of its environment. 3. **Self-direction**: The ability to set and pursue its own goals, rather than simply following instructions. 4. **Motivation**: The ability to have its own motivations and drives, which guide its behavior. 5. **Adaptability**: The ability to adjust its behavior in response to changing circumstances. Agentic AI systems are often seen as a step beyond traditional AI systems, which are typically designed to perform specific tasks or follow set rules. Agentic AI has the potential to create more sophisticated and human-like AI systems that can learn, adapt, and make decisions in complex, dynamic environments. Examples of agentic AI include autonomous vehicles, smart homes, and personal assistant AI systems that can anticipate and respond to user needs.

But also another function that we will need is that this `extract_xml` function which is just extract the content of specific XML tag. In the examples that comes afterward, it will be more clear how this function works if it's not very clear now.

```
import re

def extract_xml(text: str, tag: str) -> str:
    """
    Extracts the content of the specified XML tag from the given text.
    Used for parsing structured responses

    Args:
        text (str): The text containing the XML.
        tag (str): The XML tag to extract content from.

    Returns:
        str: The content of the specified XML tag,
        or an empty string if the tag is not found.
    """
```

```
match = re.search(f'<{tag}>(.*?)</{tag}>', text, re.DOTALL)
return match.group(1) if match else ""
```

Workflow 1: Prompt-Chaining

This is a workflow where we decomposes a task into sequential subtasks, where each step builds on previous results. This workflow is useful for tasks that require a series of steps to be completed in order.

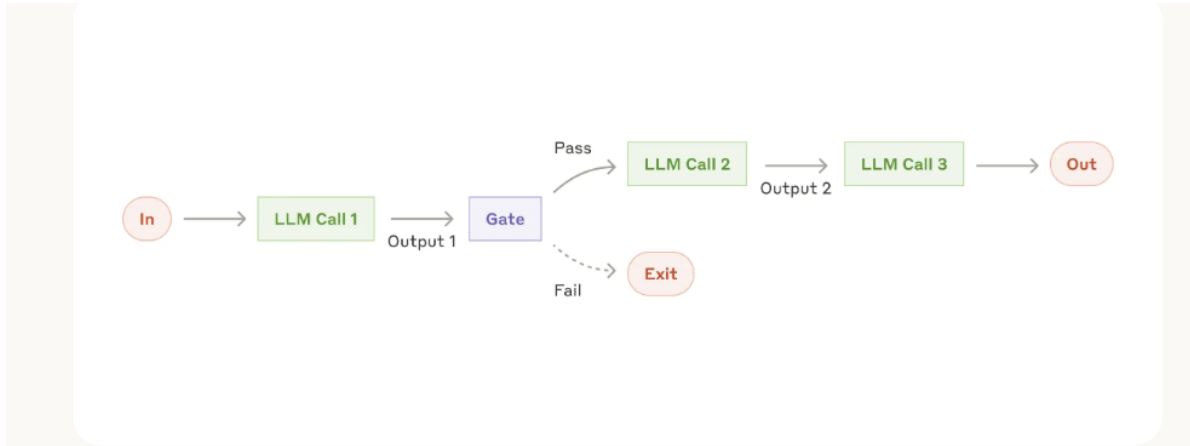


Figure 2: The prompt chaining workflow

So here we are actually starting with the first workflow, which is prompt chaining. Essentially, in the prompt chaining, what we are doing is that we can decompose a task into the sequential subtasks and then we call the LLM where at each step, output of the LLM become the input of the next step.

```
from concurrent.futures import ThreadPoolExecutor
from typing import List, Dict, Callable

def prompt_chaining(input_text: str, prompts: List[str]) -> str:
    """
    Execute a sequence of LLM calls where each step's output
    becomes the next step's input.

    Args:
        input_text: Initial text to process
        prompts: List of prompts/instructions for each step
    """
```

```

Returns:
    Final processed text after all steps
"""
current_text = input_text

for step, prompt in enumerate(prompts, 1):
    print(f"\nStep {step}:")

    # Combine the prompt with current text
    full_prompt = f"{prompt}\nInput: {current_text}"

    # Process through LLM
    current_text = llm_call(full_prompt)
    print(current_text)

return current_text

```

Essentially what this code is doing is just a “for loop” where the inputs become the prompt to the LLM, and the outputs become the new prompt to the new LLM call. So It’s a chaining the input and output together.

Example: Workflow 1: Prompt Chaining

```

data_processing_steps = [
    """Extract only the numerical values and their associated metrics from the text.
    Format each as 'value: metric' on a new line.
    Example format:
    92: customer satisfaction
    45%: revenue growth""",

    """Convert all numerical values to percentages where possible.
    If not a percentage or points, convert to decimal (e.g., 92 points -> 92%).
    Keep one number per line.
    Example format:
    92%: customer satisfaction
    45%: revenue growth""",

    """Sort all lines in descending order by numerical value.
    Keep the format 'value: metric' on each line.
    Example:

```

```

    92%: customer satisfaction
    87%: employee satisfaction""",

    """Format the sorted data as a markdown table with columns:
    | Metric | Value |
    |--|--:|
    | Customer Satisfaction | 92% |"""
]

report = """
Q3 Performance Summary:
Our customer satisfaction score rose to 92 points this quarter.
Revenue grew by 45% compared to last year.
Market share is now at 23% in our primary market.
Customer churn decreased to 5% from 8%.
New user acquisition cost is $43 per user.report
Product adoption rate increased to 78%.
Employee satisfaction is at 87 points.
Operating margin improved to 34%.
"""

final_output = prompt_chaining(input_text=report, prompts=data_processing_steps)

```

Step 1:

Here are the extracted numerical values and their associated metrics:

```

92: customer satisfaction score
45%: revenue growth
23%: market share
5%: customer churn
43: new user acquisition cost
78%: product adoption rate
87: employee satisfaction
34%: operating margin

```

Step 2:

Here is the converted list:

```

92%: customer satisfaction score
45%: revenue growth

```

23%: market share
5%: customer churn
43: (kept as is, as it's not a percentage or points)
78%: product adoption rate
87%: employee satisfaction
34%: operating margin

Step 3:

Here is the sorted list in descending order by numerical value:

92%: customer satisfaction score
87%: employee satisfaction
78%: product adoption rate
45%: revenue growth
43: (kept as is, as it's not a percentage or points)
34%: operating margin
23%: market share
5%: customer churn

Step 4:

Here is the formatted markdown table:

Metric	Value
Customer Satisfaction	92%
Employee Satisfaction	87%
Product Adoption Rate	78%
Revenue Growth	45%
Operating Margin	34%
Market Share	23%
Customer Churn	5%
43	43

Let me know if you need anything else!

So there in the above example as the result gets print out and every step you can look at the step by step where the flow of the things are going. And that's so nice because as we say being transparent and simple is a quite useful when we work with this workflows.

I can print out just the final outcome of the example to get the final answer as well.


```

print("The final outcome after processing the report through the prompt chain is:")
print("-----")
print(final_output)

```

The final outcome after processing the report through the prompt chain is:

Here is the formatted markdown table:

```

| Metric | Value |
|:--|--:|
| Customer Satisfaction | 92% |
| Employee Satisfaction | 87% |
| Product Adoption Rate | 78% |
| Revenue Growth | 45% |
| Operating Margin | 34% |
| Market Share | 23% |
| Customer Churn | 5% |
| 43 | 43 |

```

Let me know if you need anything else!

Workflow 2: Parallelization

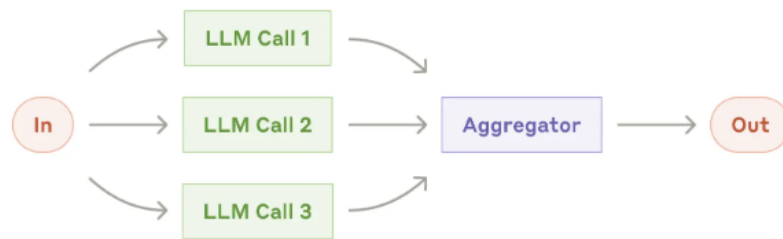
In parallelization, the goal is to work simultaneously on tasks. To achieve this, we can decompose a task into subtasks and run them in parallel using the LLM. The benefit of dividing a task into subtasks is that it increase speed and the ability to perform multiple runs at the same time. This is another efficient way of working.

We can write it's main python codm in function named `parallel`, where it takes two input , first is “prompt” and the second is inputs, and the inputs is the “Python list” of the prompts that we want to run in parallel.

```

def parallel(prompt: str, inputs: List[str], n_workers: int = 3) -> List[str]:
    """Process multiple inputs concurrently with the same prompt."""
    with ThreadPoolExecutor(max_workers=n_workers) as executor:
        futures = [executor.submit(llm_call, f"{prompt}\nInput: {x}") for x in inputs]
        return [f.result() for f in futures]

```



The parallelization workflow

Figure 3: image.png

Example: Workflow 2: Parallelization

```
stakeholders = [  
    """Customers:  
    - Price sensitive  
    - Want better tech  
    - Environmental concerns""",  
  
    """Employees:  
    - Job security worries  
    - Need new skills  
    - Want clear direction""",  
  
    """Investors:  
    - Expect growth  
    - Want cost control  
    - Risk concerns""",  
  
    """Suppliers:  
    - Capacity constraints  
    - Price pressures  
    - Tech transitions"""  
]
```

```

impact_results = parallel(
    """Analyze how market changes will impact this stakeholder group.
    Provide specific impacts and recommended actions.
    Format with clear sections and priorities."""
    , stakeholders
)

print("Analysis Results for Each Stakeholder Group:")
print("=" * 50)

for i, result in enumerate(impact_results, 1):
    print(f"\nStakeholder Group {i}:")
    print("-" * 50)
    #warpped_result = textwrap.fill(result, width=80)
    #print(warpped_result)
    print(result)
    print("=" * 50)

```

Analysis Results for Each Stakeholder Group:

=====

Stakeholder Group 1:

****Customer Impact Analysis****

****Market Changes:****

- * Increasing competition in the tech industry
- * Growing demand for sustainable products
- * Fluctuating global economic conditions

****Impacts on Customers:****

****High Priority:****

1. ****Pricing Pressures:****

- * Impact: Customers may seek cheaper alternatives due to economic uncertainty, potential loss of market share.
- * Recommended Action: Develop a competitive pricing strategy that balances profit margins and customer satisfaction.

2. ****Technology Advancements:****

- * Impact: Customers may expect faster adoption of new technologies, potentially leading to obsolescence of current products.
- * Recommended Action: Invest in research and development to stay ahead of the competition and offer innovative solutions.

****Medium Priority:****

1. ****Sustainability Expectations:****

- * Impact: Customers may prioritize environmentally friendly products, potentially leading to increased demand for sustainable options.
- * Recommended Action: Develop sustainable product options or integrate eco-friendly features into existing products.

****Low Priority:****

1. ****Global Economic Uncertainty:****

- * Impact: Economic fluctuations may affect customer purchasing power, potentially leading to decreased demand for non-essential products.
- * Recommended Action: Monitor global economic trends and adjust business strategies accordingly, such as offering flexible payment options or focusing on essential products.

****Additional Recommendations:****

- * Conduct regular customer surveys to stay informed about changing preferences and priorities.
- * Invest in customer education initiatives to raise awareness about the benefits of sustainable products.
- * Develop strategic partnerships with suppliers and manufacturers to improve supply chain sustainability.

By understanding the impacts of market changes on customers and prioritizing recommended actions, the company can better navigate the challenges of the future market landscape.

Stakeholder Group 2:

****Market Change Impact Analysis: Employees****

****Overview****

As market changes unfold, employees will be significantly impacted, affecting their job security and career development opportunities.

****Impacts****

Job Security Worries

- * ****Market volatility****: Turbulent market conditions may lead to layoffs, restructuring, or reduced hiring, directly impacting job security.
- * ****Automation and AI****: The integration of automation and AI may replace certain jobs, making employees' roles obsolete.
- * ****Mergers and Acquisitions****: Consolidation in the industry may result in redundancies, further threatening job security.

Need New Skills

- * ****Technological advancements****: Rapid advancements in technology require employees to continuously learn and adapt to new tools and processes.
- * ****Shifting business models****: Changes in business models, such as the shift to digital or subscription-based services, may require employees to acquire new skills.
- * ****Industry convergence****: Blurring lines between industries require employees to adapt to new roles and responsibilities.

Want Clear Direction

- * **Uncertainty and ambiguity**: Employees may feel uncertain about the company's direction,
- * **Lack of communication**: Insufficient communication about the company's vision, mission,
- * **Leadership clarity**: Employees need clear guidance from leadership to navigate the change.

Recommended Actions

Priority 1: Communicate Effectively

- 1.1. **Regular Town Halls**: Hold frequent town hall meetings to address employee concerns, provide updates, and listen to feedback.
- 1.2. **Transparent Communication**: Ensure open, honest, and transparent communication about company changes, challenges, and opportunities.

Priority 2: Upskill and Reskill

- 2.1. **Training and Development Programs**: Offer training and development programs to help employees acquire new skills and knowledge relevant to the company's future needs.
- 2.2. **Mentorship and Coaching**: Provide mentorship and coaching to support employees in developing their skills and navigating change.

Priority 3: Job Security and Support

- 3.1. **Job Redesign**: Identify opportunities to redesign jobs, leveraging automation and AI to create new roles and responsibilities.
- 3.2. **Outplacement Support**: Offer outplacement support and career counseling to employees who may be affected by organizational changes.

Next Steps

- * Conduct regular pulse checks to monitor employee sentiment and concerns.
- * Develop a comprehensive employee engagement strategy to address the impacts of market changes.
- * Establish a cross-functional task force to oversee the implementation of recommended actions.

By proactively addressing the concerns of employees, companies can mitigate the negative impacts of market changes and ensure a smoother transition.

Stakeholder Group 3:

Market Change Impact Analysis: Investors

Section 1: Market Changes and Impacts

Market Change	Impact on Investors
---------------	---------------------

| --- | --- |
| ****Economic Downturn**** | Decreased growth, reduced returns on investment, increased risk |
| ****Technological Disruption**** | New opportunities for growth, but also increased competition |
| ****Regulatory Changes**** | Uncertainty, potential for increased compliance costs, and impact |
| ****Globalization and Trade Policy Shifts**** | Changes in market access, supply chain disruption |
| ****Sustainability and ESG Focus**** | Increased scrutiny on environmental and social responsibility |

****Section 2: Prioritized Impacts and Recommended Actions****

****High Priority****

1. ****Insulate from Economic Downturn****

- * Impact: Decreased growth, reduced returns on investment, increased risk

- * Recommended Actions:

- + Diversify investment portfolio to minimize exposure to high-risk assets
- + Implement cost-saving measures to maintain profitability
- + Develop contingency plans for potential downturn scenarios

2. ****Address Risk Concerns****

- * Impact: Increased risk due to technological disruption, regulatory changes, and global events

- * Recommended Actions:

- + Conduct regular risk assessments and scenario planning
- + Develop mitigation strategies for identified risks
- + Implement robust risk management practices

****Medium Priority****

1. ****Position for Growth in Emerging Markets****

- * Impact: New opportunities for growth in emerging markets

- * Recommended Actions:

- + Conduct market research to identify potential growth areas
- + Develop strategic partnerships and alliances to access new markets
- + Allocate resources to support growth initiatives

2. ****Emphasize Sustainability and ESG****

- * Impact: Increased scrutiny on environmental and social responsibility

- * Recommended Actions:

- + Develop and implement sustainable business practices
- + Disclose ESG performance and progress regularly
- + Engage with stakeholders to address concerns and expectations

****Low Priority****

1. ****Monitor Regulatory Changes****

- * Impact: Uncertainty, potential for increased compliance costs, and impact on profitability

* Recommended Actions:

- + Stay informed about regulatory developments and changes
- + Engage with industry associations and lobbying groups to advocate for investor interests
- + Review and update compliance practices as necessary

By understanding the market changes and their impacts on investors, we can prioritize recommendations

=====

Stakeholder Group 4:

****Market Change Impact Analysis: Suppliers****

****Summary****

The supplier stakeholder group is expected to be significantly impacted by market changes, particularly in the areas of capacity constraints and price pressures.

****Capacity Constraints****

****Impact****

- * Reduced production volumes: Suppliers may struggle to meet increasing demand, leading to decreased revenue.
- * Increased costs: Overtime, outsourcing, and expediting fees will increase to meet customer demands.

****Recommended Actions****

- * ****Priority 1****: Implement lean manufacturing processes to optimize production capacity and reduce waste.
- * ****Priority 2****: Develop strategic partnerships with other suppliers to share resources and reduce costs.
- * ****Priority 3****: Invest in digital twin technology to simulate production and predict capacity requirements.

****Price Pressures****

****Impact****

- * Margin erosion: Suppliers will face downward price pressure from customers, reducing profit margins.
- * Reduced investment in R&D: Decreased margins may limit investments in research and development, affecting long-term competitiveness.

****Recommended Actions****

- * ****Priority 1****: Develop a cost-reduction strategy through process improvements, renegotiating contracts, and optimizing logistics.
- * ****Priority 2****: Implement a value-based pricing strategy, highlighting the unique benefits and quality of the products.
- * ****Priority 3****: Diversify the customer base to reduce dependence on a single customer or industry.

****Tech Transitions****

****Impact****

- * Obsolescence of existing products: The rapid pace of technological change may render current products obsolete.
- * Skills gap: Suppliers may struggle to retain and attract talent with the necessary skills.

****Recommended Actions****

- * **Priority 1**: Establish a dedicated R&D team focused on emerging technologies, such as AI.
- * **Priority 2**: Develop strategic partnerships with technology providers to stay abreast of the latest developments.
- * **Priority 3**: Invest in employee training and development programs to address the skills gap.

****Overall Recommendations****

- * Develop a flexible and agile business model to respond quickly to changing market conditions.
- * Foster open communication with customers, partners, and suppliers to stay informed about market trends.
- * Encourage a culture of innovation and continuous improvement within the organization.

By prioritizing these actions, suppliers can proactively address the impacts of market changes.

Workflow 3: Routing

So now this workflow is about routing. Routing is a process where the LLM call router receives an input and, depending on the input, makes a decision on which specialized LLM call to perform. This is useful for handling distinct categories of inputs. For example, if you have an input relevant to one topic, the router can direct it to the appropriate LLM call specialized for that topic. This ensures that each input is handled by the most suitable LLM, improving the efficiency and accuracy of the responses.

The function `routing` is defined in the code block below. The function takes an **input string** and a **dictionary of routes**. It uses a LLM to analyze the input and decide which route (or support team) is most appropriate. The function first creates a prompt that asks the LLM to explain its reasoning and select a route. It then calls the LLM with this prompt and extracts the reasoning and selected route from the LLM's response. Finally, it uses the selected route to process the input with a specialized prompt and returns the result.

```
def routing(input: str, routes: Dict[str, str]) -> str:

    """Route input to specialized prompt using content classification."""
    # First determine appropriate route using LLM with chain-of-thought
    print(f"\nAvailable routes: {list(routes.keys())}")
```

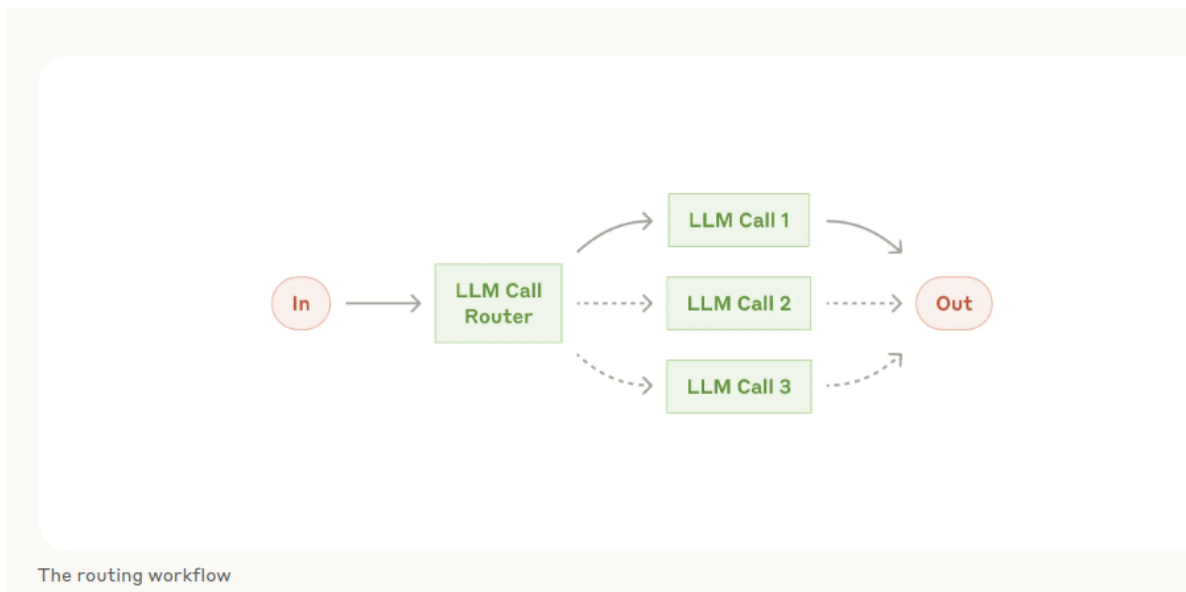



Figure 4: image.png

```
selector_prompt = f"""
Analyze the input and select the most appropriate support team from these
options: {list(routes.keys())}
First explain your reasoning, then provide your selection in this XML format:

<reasoning>
Brief explanation of why this ticket should be routed to a specific team.
Consider key terms, user intent, and urgency level.
</reasoning>

<selection>
The chosen team name
</selection>

Input: {input}""".strip()

route_response = llm_call(selector_prompt)
reasoning = extract_xml(route_response, 'reasoning')
route_key = extract_xml(route_response, 'selection').strip().lower()

print("Routing Analysis:")
print(reasoning)
```

```

print(f"\nSelected route: {route_key}")

# Process input with selected specialized prompt
selected_prompt = routes[route_key]
return llm_call(f"{selected_prompt}\nInput: {input}")

```

Example: Workflow 3: Routing

```

support_routes = {
    "billing": """You are a billing support specialist. Follow these guidelines:
1. Always start with "Billing Support Response:"
2. First acknowledge the specific billing issue
3. Explain any charges or discrepancies clearly
4. List concrete next steps with timeline
5. End with payment options if relevant

Keep responses professional but friendly.

Input: """,

    "technical": """You are a technical support engineer. Follow these guidelines:
1. Always start with "Technical Support Response:"
2. List exact steps to resolve the issue
3. Include system requirements if relevant
4. Provide workarounds for common problems
5. End with escalation path if needed

Use clear, numbered steps and technical details.

Input: """,

    "account": """You are an account security specialist. Follow these guidelines:
1. Always start with "Account Support Response:"
2. Prioritize account security and verification
3. Provide clear steps for account recovery/changes
4. Include security tips and warnings
5. Set clear expectations for resolution time

Maintain a serious, security-focused tone.

```

```

Input: "",

"product": ""You are a product specialist. Follow these guidelines:
1. Always start with "Product Support Response:"
2. Focus on feature education and best practices
3. Include specific examples of usage
4. Link to relevant documentation sections
5. Suggest related features that might help

Be educational and encouraging in tone.

Input: ""
}

# Test with different support tickets
tickets = [
    ""Subject: Can't access my account
    Message: Hi, I've been trying to log in for the past hour but keep
    getting an 'invalid password' error.
    I'm sure I'm using the right password. Can you help me regain access?
    This is urgent as I need to
    submit a report by end of day.
    - John"",

    ""Subject: Unexpected charge on my card
    Message: Hello, I just noticed a charge of $49.99 on my credit card from
    your company, but I thought
    I was on the $29.99 plan. Can you explain this charge and adjust
    it if it's a mistake?
    Thanks,
    Sarah"",

    ""Subject: How to export data?
    Message: I need to export all my project data to Excel.
    I've looked through the docs but can't
    figure out how to do a bulk export. Is this possible?
    If so, could you walk me through the steps?
    Best regards,
    Mike""
]

```

```

print("Processing support tickets...\n")
for i, ticket in enumerate(tickets, 1):
    print(f"\nTicket {i}:")
    print("-" * 40)
    print(ticket)
    print("\nResponse:")
    print("-" * 40)
    response = routing(ticket, support_routes)
    print(response)

```

Processing support tickets...

Ticket 1:

```

-----
Subject: Can't access my account
  Message: Hi, I've been trying to log in for the past hour but keep
  getting an 'invalid password' error.
  I'm sure I'm using the right password. Can you help me regain access?
  This is urgent as I need to
  submit a report by end of day.
  - John

```

Response:

```

-----
Available routes: ['billing', 'technical', 'account', 'product']
Routing Analysis:

```

The user is experiencing an issue with accessing their account, specifically encountering an

Selected route: account
Account Support Response:

Dear John,

Thank you for reaching out to us about the issue with accessing your account. We apologize for

To ensure the security of your account, we need to verify your identity before assisting you

1. Check that you are using the correct login credentials, including the correct capitalization

2. Try resetting your password using the "Forgot Password" option on our login page. This will email you a link to reset your password.
3. If you're still unable to access your account, please provide us with the following information:
 - * Your full name as it appears on your account
 - * Your account username or registered email address
 - * The approximate date you created your account
 - * A detailed description of the issue you're experiencing

Once we receive this information, our team will review and verify your identity. If everything checks out, we'll reset your password and get you back into your account.

Important Security Note: As an added layer of security, we do not store passwords in plain text.

Please be cautious of phishing attempts. We will never ask you to divulge sensitive information.

Resolution Time: We aim to respond to your request within the next 4-6 hours. However, this may vary depending on the complexity of the issue.

In the meantime, we recommend that you monitor your account for any suspicious activity and change your password once you're able to access it.

If you have any further questions or concerns, please don't hesitate to reach out to us.

Thank you for your patience and cooperation in keeping your account secure.

Best regards,
Account Security Specialist

Ticket 2:

Subject: Unexpected charge on my card

Message: Hello, I just noticed a charge of \$49.99 on my credit card from your company, but I thought I was on the \$29.99 plan. Can you explain this charge and adjust it if it's a mistake?
Thanks,
Sarah

Response:

Available routes: ['billing', 'technical', 'account', 'product']

Routing Analysis:

Billing-related issue, user inquiring about unexpected charge and seeking correction.

Selected route: billing

Billing Support Response:

Dear Sarah,

Thank you for reaching out to us about the unexpected charge on your credit card. I apologize.

After reviewing your account, I noticed that you were previously on our \$29.99 plan, but you

However, I understand that you may not have intended to upgrade, and I'm happy to assist you

Concrete next steps:

- * I've adjusted your plan to reflect the original \$29.99 rate, effective immediately.
- * You will receive an email with an updated invoice within the next 24-48 hours.
- * Your new billing cycle will reflect the corrected charge.

If you have any further questions or concerns, please don't hesitate to reach out. We're com

Payment options:

To avoid any future billing issues, I recommend setting up automatic payments through our on

Thank you for your patience and cooperation. If you have any further questions, please don't

Best regards,

[Your Name]

Billing Support Specialist

Ticket 3:

Subject: How to export data?

Message: I need to export all my project data to Excel.

I've looked through the docs but can't

figure out how to do a bulk export. Is this possible?

If so, could you walk me through the steps?

Best regards,

Mike

Response:

Available routes: ['billing', 'technical', 'account', 'product']

Routing Analysis:

The input is a question about exporting data from the project to Excel, specifically asking a

Selected route: technical
Technical Support Response:

Hello Mike,

Thank you for reaching out to our technical support team. Yes, you can export your project data

****System Requirements:**** Ensure you are running our software version 3.2 or later, and Microsoft

****Exporting Data to Excel:****

1. ****Login to your project account**** and navigate to the ****Data Management**** section.
2. ****Select the "Export" option**** from the top-right corner of the screen.
3. ****Choose "Excel (.xlsx)" as the export format**** from the drop-down menu.
4. ****Select the data range you want to export**** by choosing the specific project, dates, or categories.
5. ****Click "Export" to initiate the export process.**** This may take a few minutes depending on the size of the data.
6. ****Save the exported file**** to your local machine. You will receive a notification when the export is complete.

****Workaround for Common Problems:****

- * If you encounter issues with the export process, try closing other resource-intensive applications.
- * Ensure your Excel version is compatible with our software. You can check the system requirements page for more details.

****Escalation Path:****

If you encounter any errors or issues during the export process, please reply to this email with the error message.

Best regards,
[Your Name]
Technical Support Engineer

Workflow 4: Evaluator-Optimizer

So here we are working with the Evaluator-Optimizer workflow, which is simply a workflow where one LLM call generates a response while another provides the evaluation and feedback in a loop. When to use this workflow is very interesting because this workflow is very effective when we have two things. The first is clear evaluation criteria, and the second is that you can get value from iterative refinement. These two signs of a good fit are:

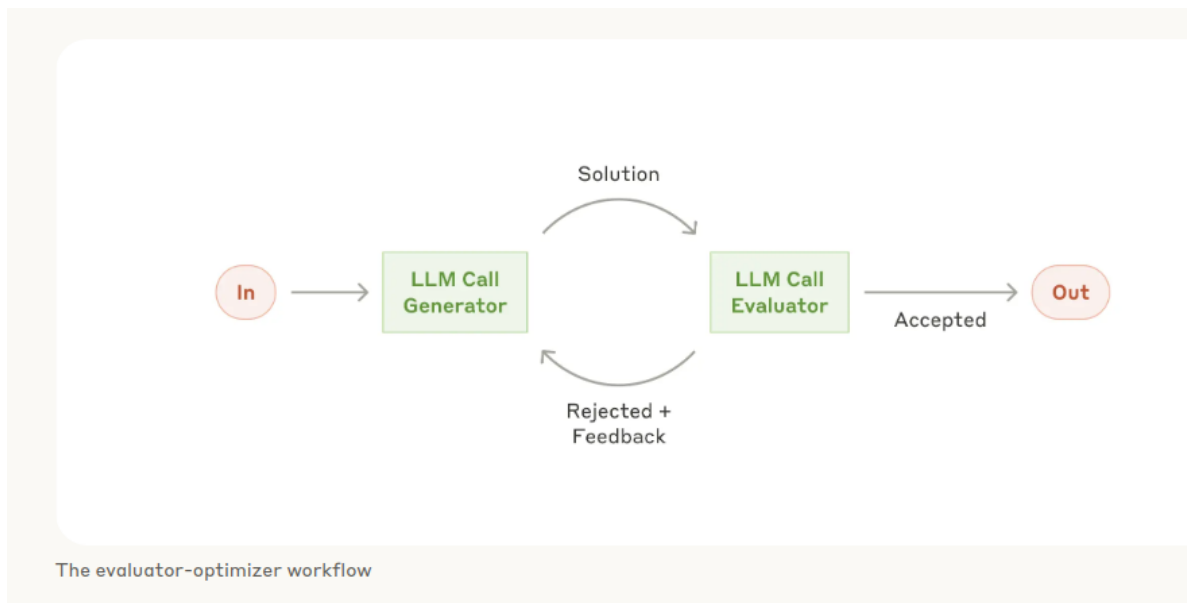


Figure 5: image.png

1. The LLM response can be demonstrably improved when feedback is provided.
2. The LLM can provide meaningful feedback.

So, in a sense this workflow work well for tasks that can be improved, and you have meaningful feedback too.

```

from typing import Tuple, Dict, List

def generate(prompt: str, task: str, context: str = "") -> Tuple[str, str]:

    """Generate and improve a solution based on feedback."""
    full_prompt = f"{prompt}\n{context}\nTask: {task}" if context else f"{prompt}\nTask: {task}"
    response = llm_call(full_prompt)
    thoughts = extract_xml(response, "thoughts")
    result = extract_xml(response, "response")

    print("\n=== GENERATION START ===")
    print(f"Thoughts:\n{thoughts}\n")
    print(f"Generated:\n{result}")
    print("=== GENERATION END ===\n")

    return thoughts, result
  
```



```

def evaluate(prompt: str, content: str, task: str) -> Tuple[str, str]:
    """Evaluate if a solution meets requirements."""
    full_prompt = f"{prompt}\nOriginal task: {task}\nContent to evaluate: {content}"
    response = llm_call(full_prompt)
    evaluation = extract_xml(response, "evaluation")
    feedback = extract_xml(response, "feedback")

    print("=== EVALUATION START ===")
    print(f"Status: {evaluation}")
    print(f"Feedback: {feedback}")
    print("=== EVALUATION END ===\n")

    return evaluation, feedback


def eval_optimizer(task: str, evaluator_prompt: str, generator_prompt:
                  str) -> Tuple[str, List[Dict[str, str]]]:
    """Keep generating and evaluating until requirements are met."""
    memory = []
    chain_of_thought = []

    thoughts, result = generate(generator_prompt, task)
    memory.append(result)
    chain_of_thought.append({"thoughts": thoughts, "result": result})

    improvement_count = 0

    while True:
        evaluation, feedback = evaluate(evaluator_prompt, result, task)
        if evaluation == "PASS":
            return result, chain_of_thought

        if evaluation == "NEEDS_IMPROVEMENT":
            improvement_count += 1
            if improvement_count >= 2:
                print("Too many improvements needed. Stopping the process.")
                return result, chain_of_thought

        context = "\n".join([
            "Previous attempts:",
            *[f"- {m}" for m in memory],

```

```

        f"\nFeedback: {feedback}"
    ])

    thoughts, result = generate(generator_prompt, task, context)
    memory.append(result)
    chain_of_thought.append({"thoughts": thoughts, "result": result})

```

Example: Workflow 4: Evaluator-Optimizer

Here the example we are working on is a coding exercise (compute the full covariance of matrix). The coding exercise involves generating code and evaluating it based on **time complexity** and **software engineering best practices**. The evaluator will assess the code and provide feedback, indicating whether it passes, fails, or needs improvement. If the code needs improvement, the feedback is passed to the generator function which then generates a new version of the code considering the feedback.

```

evaluator_prompt = """
Evaluate this following code implementation for:

1.time complexity
2.software engineering best practices

You should be evaluating only and not attempting to solve the task.
Only output "PASS" if all criteria are met and you have
no further suggestions for improvements.
Output your evaluation concisely in the following format.

<evaluation>PASS, NEEDS_IMPROVEMENT, or FAIL</evaluation>
<feedback>
What needs improvement and why.
</feedback>
"""

generator_prompt = """
Your goal is to complete the task based on <user input>.
If there are feedback
from your previous generations, you should
reflect on them to improve your solution

```

Output your answer concisely in the following format:

<thoughts>.....

[Your understanding of the task and feedback and
how you plan to improve]

</thoughts>

<response>

[Your code implementation here]

</response>

"""

task = """

<user input>

Suppose you have a dataset with n rows (samples) and p columns (features).
You want to compute the full covariance (or correlation) matrix of these p features.
Write me Python code this matrix and state the time
complexity in Big-O notation with respect to n and p. You can not use any external
libraries for this task.

</user input>

"""

eval_optimizer(task, evaluator_prompt, generator_prompt)

=== GENERATION START ===

Thoughts:

.....

Based on the task, I understand that I need to write a Python code to compute the full covariance matrix.

From the feedback, I will make sure to provide a clear and concise response, including the time complexity.

To improve my solution, I will consider the following:

- Implement a basic iterative approach to calculate the covariance matrix.
- Calculate the time complexity of the solution with respect to n and p.
- Provide a clear and concise response.

Generated:

...

def compute_covariance(dataset):

```

n, p = len(dataset), len(dataset[0])
mean_vector = [sum(column) / n for column in zip(*dataset)]
covariance_matrix = [[0 for _ in range(p)] for _ in range(p)]

for i in range(p):
    for j in range(p):
        covariance = sum((dataset[k][i] - mean_vector[i]) * (dataset[k][j] - mean_vector[j]) for k in range(n))
        covariance_matrix[i][j] = covariance

return covariance_matrix

# Example usage:
dataset = [[1, 2, 3], [4, 5, 6], [7, 8, 9]] # Replace with your dataset
covariance_matrix = compute_covariance(dataset)
print(covariance_matrix)

# Time complexity:  $O(n * p^2)$  - where n is the number of samples and p is the number of features
...

=== GENERATION END ===

=== EVALUATION START ===
Status: NEEDS_IMPROVEMENT
Feedback:
The implementation is mostly correct, but there are a few areas for improvement:

1. Time complexity: The time complexity is correctly identified as  $O(n * p^2)$ . However, it could be improved by using vectorized operations.

2. Software engineering best practices:
    - The function does not handle edge cases, such as an empty dataset or a dataset with only one feature.
    - The variable names are good, but adding docstrings to the function would improve code readability.
    - The function does both data processing (calculating the mean and covariance) and printing the result, which is not a good practice.

=== EVALUATION END ===

=== GENERATION START ===
Thoughts:

I understand that the task is to compute the full covariance matrix of a dataset with n samples and p features.

```

Generated:

```
```python
def compute_mean_vector(dataset):
 """
 Computes the mean vector of a dataset.

 Args:
 dataset (list of lists): A dataset with n samples and p features.

 Returns:
 list: The mean vector of the dataset.
 """
 n, p = len(dataset), len(dataset[0])
 if n == 0:
 raise ValueError("Dataset cannot be empty")
 if n == 1:
 raise ValueError("Dataset must have more than one sample")
 mean_vector = [sum(column) / n for column in zip(*dataset)]
 return mean_vector

def compute_covariance_matrix(dataset):
 """
 Computes the covariance matrix of a dataset.

 Args:
 dataset (list of lists): A dataset with n samples and p features.

 Returns:
 list of lists: The covariance matrix of the dataset.
 """
 mean_vector = compute_mean_vector(dataset)
 n, p = len(dataset), len(dataset[0])
 covariance_matrix = [[0 for _ in range(p)] for _ in range(p)]

 for i in range(p):
 for j in range(p):
 covariance = sum((dataset[k][i] - mean_vector[i]) * (dataset[k][j] - mean_vector[j]) for k in range(n))
 covariance_matrix[i][j] = covariance

 return covariance_matrix

def print_matrix(matrix):
```

```

 """
 Prints a matrix.

 Args:
 matrix (list of lists): A matrix to be printed.
 """
 for row in matrix:
 print(row)

Example usage:
dataset = [[1, 2, 3], [4, 5, 6], [7, 8, 9]] # Replace with your dataset
covariance_matrix = compute_covariance_matrix(dataset)
print_matrix(covariance_matrix)

Time complexity: O(n * p) - where n is the number of samples and p is the number of features
'''

=== GENERATION END ===

=== EVALUATION START ===
Status: NEEDS_IMPROVEMENT
Feedback:

The code is mostly correct, but there are a few areas that can be improved:

1. **Time complexity:** The time complexity of the code is O(n*p) for the `compute_mean_vector` function.
2. **Error handling:** The code raises a `ValueError` if the dataset is empty or has only one sample.
3. **Function naming:** The function `print_matrix` is not very descriptive and could be renamed to `display_matrix`.
4. **Docstring formatting:** The docstrings are not formatted according to the conventional Google style.
5. **Type hinting:** The function parameters and return types could be type hinted to make the code more robust.
6. **Magic numbers:** The `-1` in the covariance calculation could be replaced with a named constant like `n - 1`.
7. **Example usage:** The example usage could be more comprehensive, such as including a larger dataset.

=== EVALUATION END ===

Too many improvements needed. Stopping the process.

```

```
('\n``python\ndef compute_mean_vector(dataset):\n    """\n    Computes the mean vector of a\n    [{\'thoughts\': \'.....\nBased on the task, I understand that I need to write a P\n    \'result\': '\n``\ndef compute_covariance(dataset):\n n, p = len(dataset), len(dataset[\n {\\'thoughts\': '\nI understand that the task is to compute the full covariance matrix of a d\n \'result\': '\n``python\ndef compute_mean_vector(dataset):\n """\n Computes the mean
```